

✓ Credit Card Fraud

Feature Explanation:

distance_from_home - the distance from home where the transaction happened.

distance_from_last_transaction - the distance from last transaction happened.

ratio_to_median_purchase_price - Ratio of purchased price transaction to median purchase price.

repeat_retailer - Is the transaction happened from same retailer.

used_chip - Is the transaction through chip (credit card).

used_pin_number - Is the transaction happened by using PIN number.

online_order - Is the transaction an online order.

fraud - Is the transaction fraudulent. *0 the transaction is not fraudulent, 1 the transaction is fraudulent*

```
#Import libraries for analysis of the the dataset
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import kagglehub
import os
```

```
#Load Data
```

```
file_path = kagglehub.dataset_download("dhanushnarayananr/credit-card-fraud")
```

```
#Files downloaded
```

```
files = os.listdir(file_path)
print("Files in the dataset directory:", files)
```

```
🔄 Downloading from https://www.kaggle.com/api/v1/datasets/download/dhanushnarayananr/credit-card-fraud?dataset_version_number=1...
100%|██████████| 28.9M/28.9M [00:00<00:00, 49.3MB/s]
Extracting files...
Files in the dataset directory: ['card_transdata.csv']
```

```
# Select the first CSV file (assuming it exists)
```

```
csv_file = [f for f in files if f.endswith(".csv")][0] # Get the first CSV file
```

```
# Load card_transdata dataset and assign to a variable
```

```
cardtrans = pd.read_csv(os.path.join(file_path, csv_file))
```

```
# Print the first few rows
```

```
print("First 5 rows of the dataset:")
cardtrans.head()
```

```
🔄 First 5 rows of the dataset:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_price	repeat_retailer	used_chip	used_pin_number	onli
0	57.877857	0.311140	1.945940	1.0	1.0	0.0	
1	10.829943	0.175592	1.294219	1.0	0.0	0.0	
2	5.091079	0.805153	0.427715	1.0	0.0	0.0	
3	2.247564	5.600044	0.362663	1.0	1.0	0.0	
4	44.190936	0.566486	2.222767	1.0	1.0	0.0	

```
#get data discriptive statistics
cardtrans.describe()
```



	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_price	repeat_retailer	used_chip	used_pin_num
count	1000000.000000	1000000.000000	1000000.000000	1000000.000000	1000000.000000	1000000.000
mean	26.628792	5.036519	1.824182	0.881536	0.350399	0.100
std	65.390784	25.843093	2.799589	0.323157	0.477095	0.300
min	0.004874	0.000118	0.004399	0.000000	0.000000	0.000
25%	3.878008	0.296671	0.475673	1.000000	0.000000	0.000
50%	9.967760	0.998650	0.997717	1.000000	0.000000	0.000
75%	25.743985	3.355748	2.096370	1.000000	1.000000	0.000
max	10632.723672	11851.104565	267.802942	1.000000	1.000000	1.000



```
#Get data demensions
cardtrans.shape
```



```
(1000000, 8)
```

```
#Check for null values
cardtrans.isnull().sum()
```



	0
distance_from_home	0
distance_from_last_transaction	0
ratio_to_median_purchase_price	0
repeat_retailer	0
used_chip	0
used_pin_number	0
online_order	0
fraud	0

dtype: int64

```
#Check datatypes
cardtrans.dtypes
```



	0
distance_from_home	float64
distance_from_last_transaction	float64
ratio_to_median_purchase_price	float64
repeat_retailer	float64
used_chip	float64
used_pin_number	float64
online_order	float64
fraud	float64

dtype: object

```
#Get the distribution of values for 0 and 1
fraud_counts = cardtrans['fraud'].value_counts()
fraud_counts
```



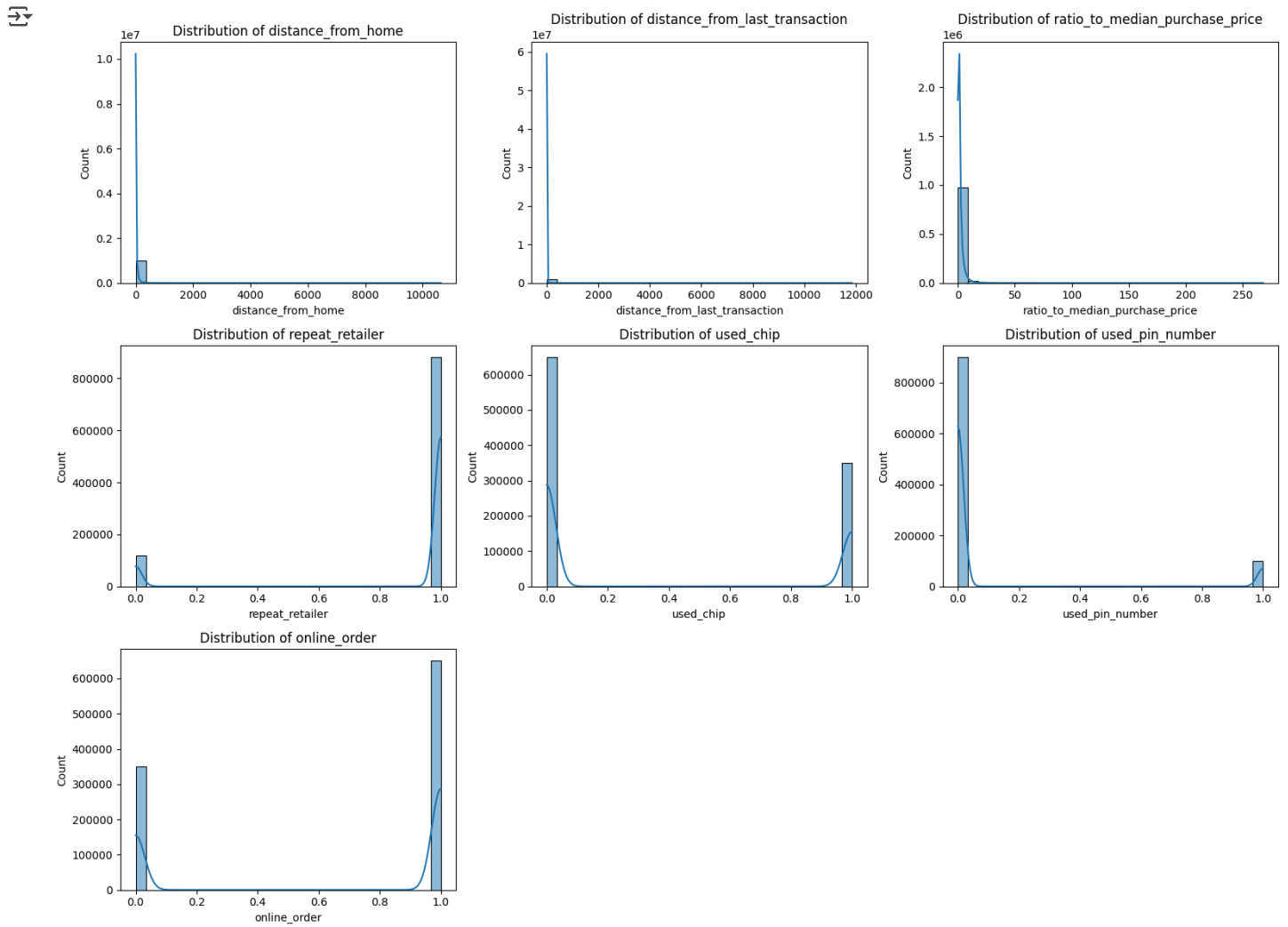
	count
0.0	912597
1.0	87403

dtype: int64

▼ Data Visualizations

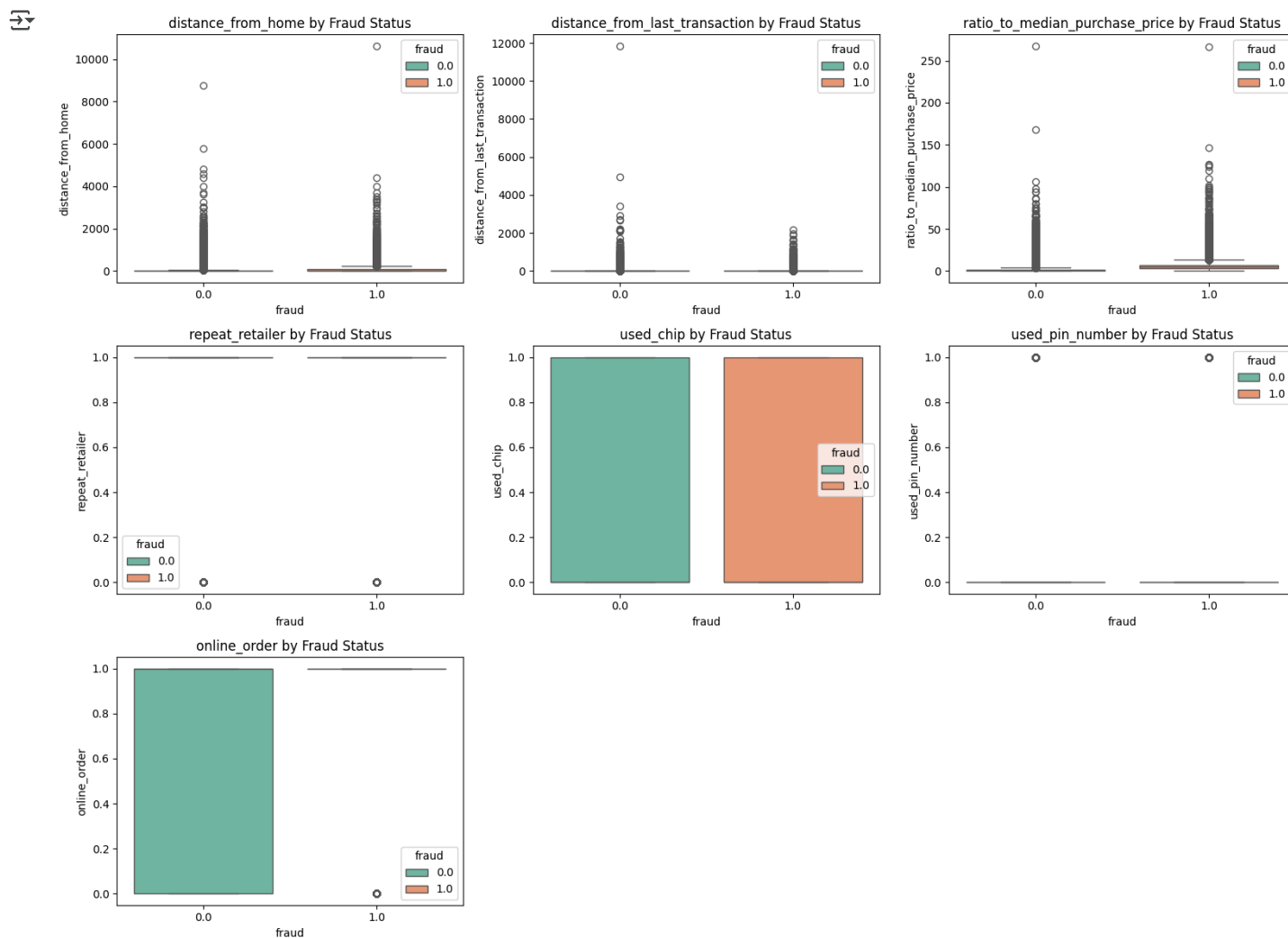
Histograms

```
#Histograms for all numerical features
plt.figure(figsize=(16,12))
for i, col in enumerate(cardtrans.drop(columns="fraud").columns):
    plt.subplot(3, 3, i + 1)
    sns.histplot(cardtrans[col], kde=True, bins=30)
    plt.title(f"Distribution of {col}")
plt.tight_layout()
plt.show()
```



Boxplots

```
#Boxplot of all features
plt.figure(figsize=(16, 12))
for i, col in enumerate(cardtrans.drop(columns='fraud').columns):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(x='fraud', y=col, data=cardtrans, palette="Set2", hue = 'fraud')
    plt.title(f"{col} by Fraud Status")
plt.tight_layout()
plt.show()
```



Show boxplots of continuous variables

Continuous variables are distance_from_home, distance_from_last_transaction, and ratio_to_median_purchase_price

boolean variables are: repeat_retailer, nused_chip, used_pin_number, online_order, and fraud

#Boxplot of continuous variables with log+1 transformation to reduce positive skewness and improve data symmetry

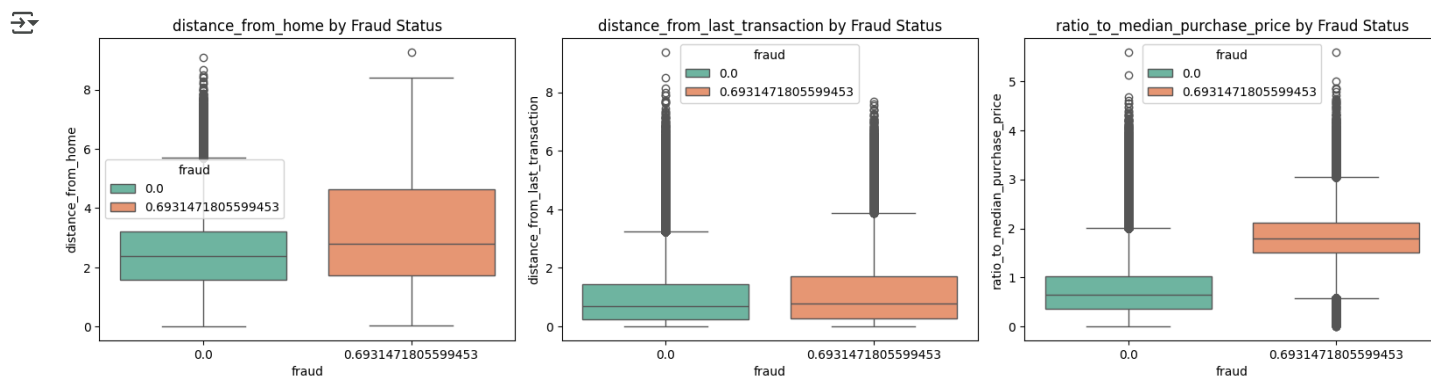
#list continuous variables

```
Cont_cols = cardtrans[['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_purchase_price', 'fraud']]
```

```

Cont_cols = np.log(Cont_cols+1)
plt.figure(figsize=(16, 12))
for i, col in enumerate(Cont_cols.drop(columns= ('fraud')).columns):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(x='fraud', y=col, data=Cont_cols, palette="Set2", hue = 'fraud')
    plt.title(f"{col} by Fraud Status")
plt.tight_layout()
plt.show()

```



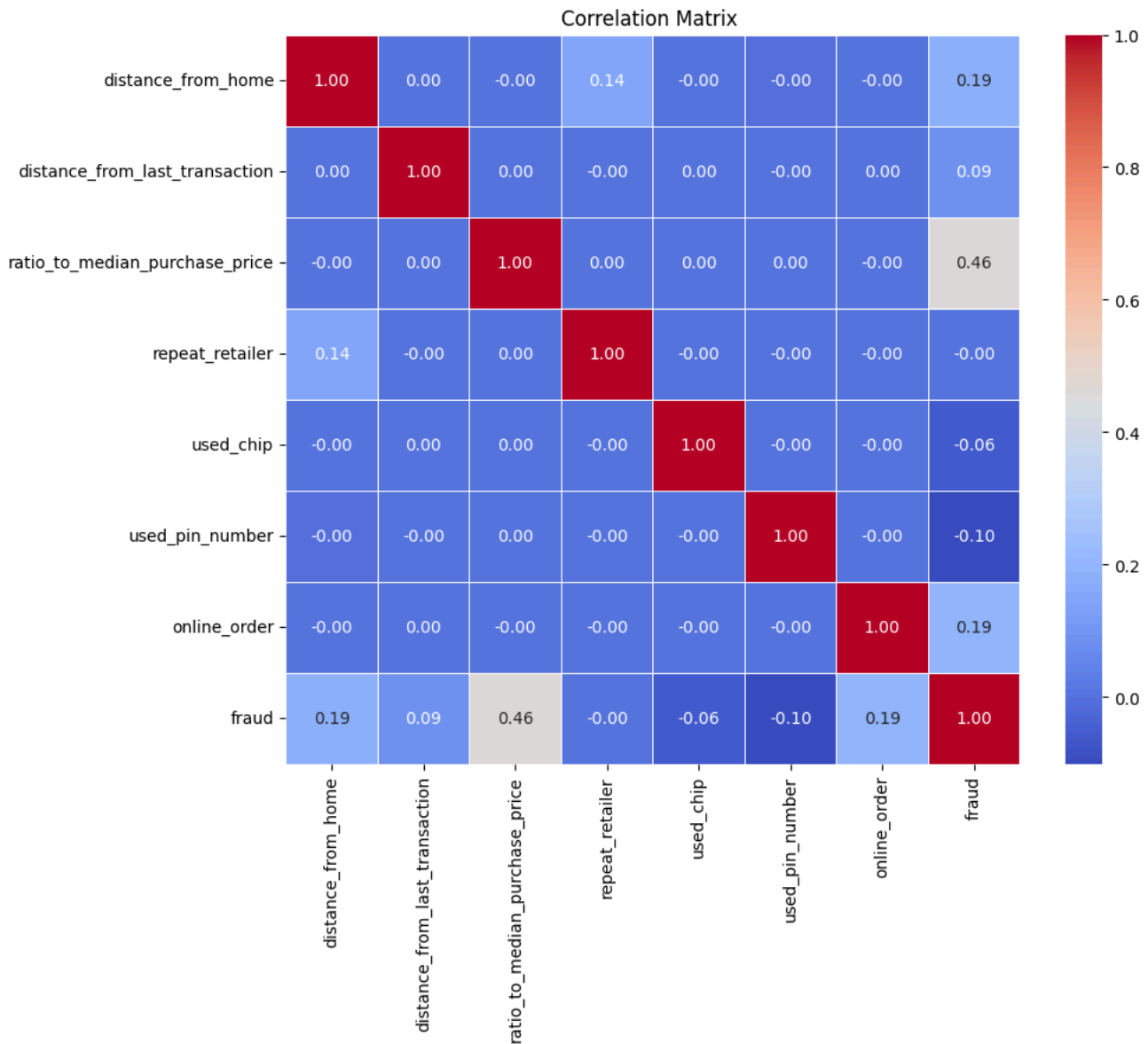
Correlation Matrix

```

# Compute correlation matrix
corr_matrix = cardtrans.corr()

#plot heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Matrix ")
plt.show()

```



Create a triangle heatmap

Cytation for reference page

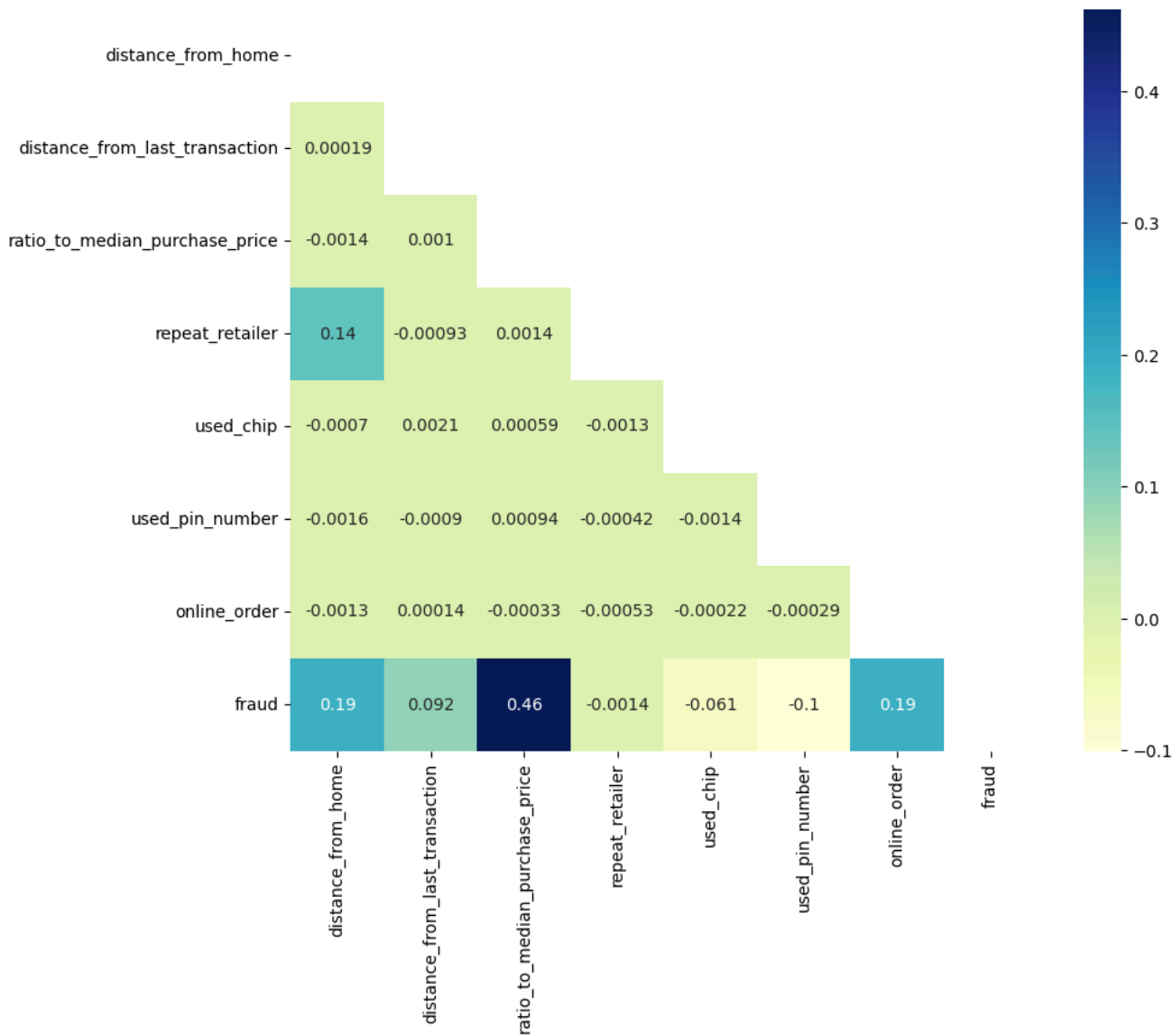
Geeks for Geeks. (2024). How to create a Triangle Correlation Heatmap in seaborn – Python?. *Geeks for Geeks*.

<https://www.geeksforgeeks.org/how-to-create-a-triangle-correlation-heatmap-in-seaborn-python/>

```
#Plot triangle heatmap

# creating mask
mask = np.triu(np.ones_like(cardtrans.corr()))

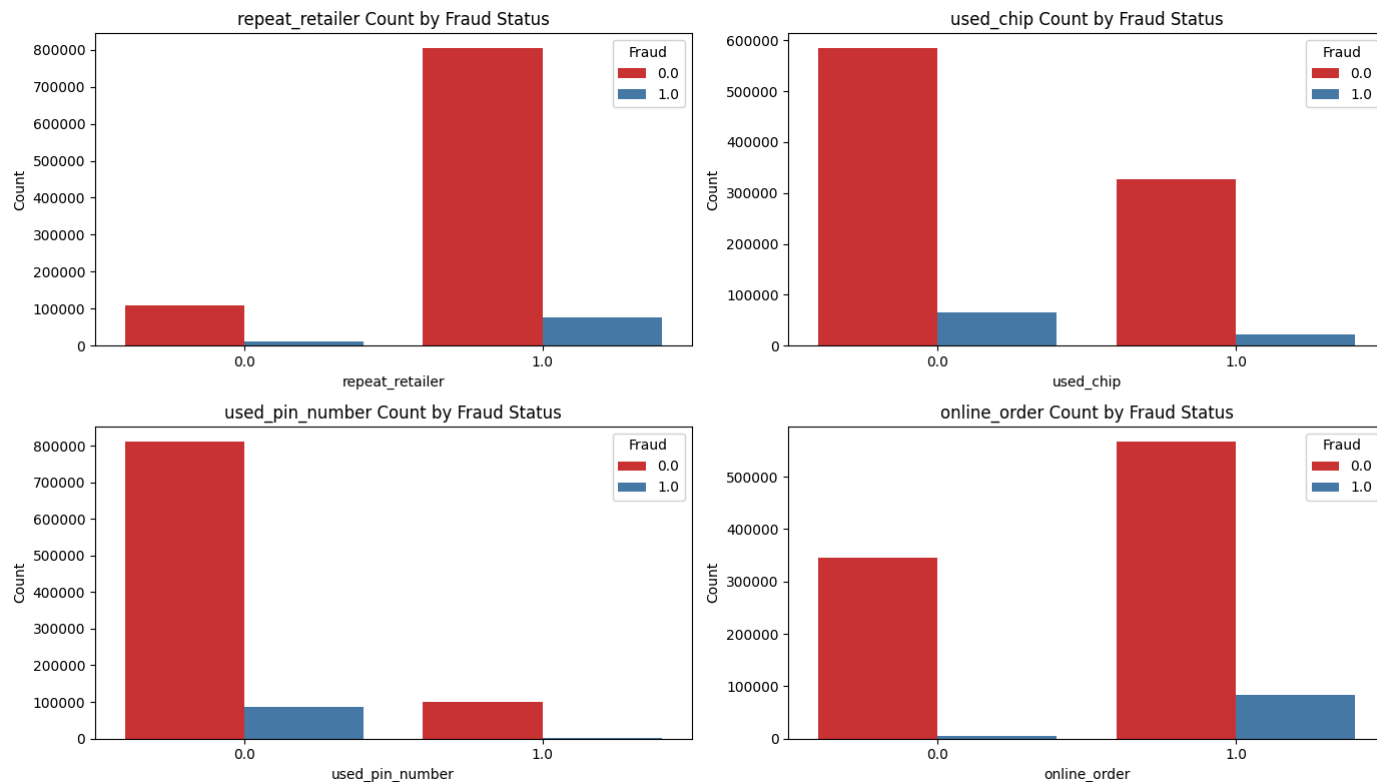
# plotting a triangle correlation heatmap
plt.figure(figsize=(10, 8))
dataplot = sns.heatmap(cardtrans.corr(), cmap="YlGnBu", annot=True, mask=mask)
```



Count Plot

```
binary_cols = ['repeat_retailer', 'used_chip', 'used_pin_number', 'online_order']
```

```
plt.figure(figsize=(14, 8))
for i, col in enumerate(binary_cols):
    plt.subplot(2, 2, i + 1)
    sns.countplot(x=col, hue='fraud', data=cardtrans, palette='Set1')
    plt.title(f"{col} Count by Fraud Status")
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.legend(title="Fraud")
plt.tight_layout()
plt.show()
```

```
print("\nEDA Summary:")
print("- Dataset has no missing values.")
print(f"- Fraud cases distribution: {fraud_counts.to_dict()}")
print("- Correlation highest between fraud and Ratio to Median Purchase Price.")
```



```
EDA Summary:
- Dataset has no missing values.
- Fraud cases distribution: {0.0: 912597, 1.0: 87403}
- Correlation highest between fraud and Ratio to Median Purchase Price.
```

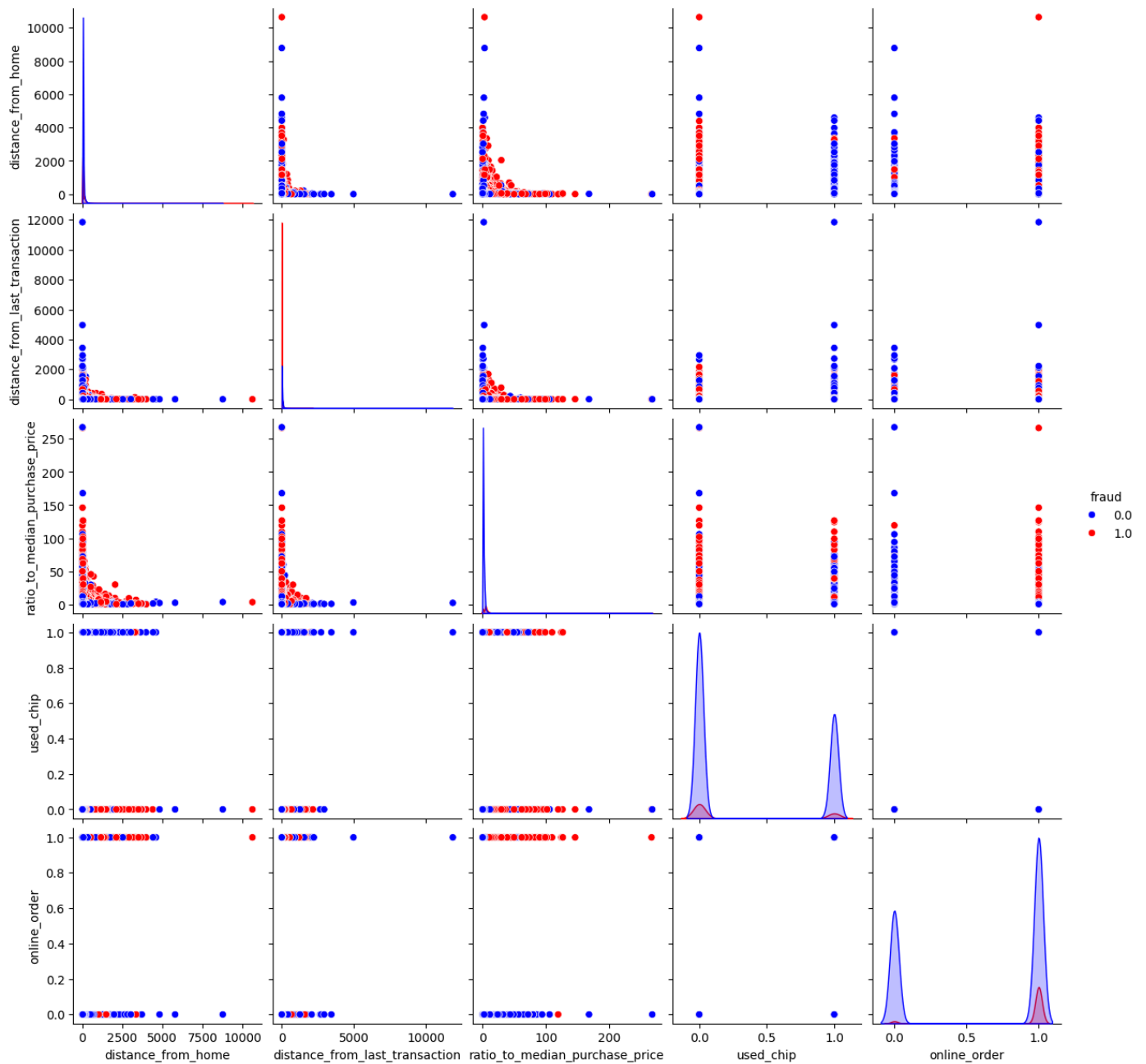
Pairplot

```
#Reduce pairplot to most informative features (avoid overloading)
important_features = ['distance_from_home', 'distance_from_last_transaction',
                     'ratio_to_median_purchase_price', 'used_chip', 'online_order', 'fraud']

sns.pairplot(cardtrans[important_features], hue='fraud', palette={0: 'blue', 1: 'red'}, diag_kind='kde')
plt.suptitle("Pairplot of Key Features Colored by Fraud", y=1.02)
plt.show()
```



Pairplot of Key Features Colored by Fraud



✓ Partition the dataset into training and testing datasets for modeling

This dataset contains 1,000,000 total cases. Since this dataset is relatively large, a partitioning between 75-90% may be acceptable. (Larose & Larose, 2019)

```
#Import data partitioning libraries
from sklearn.model_selection import train_test_split
import random

#partition the dataset into training and testing datasets
#test_size 80% partitioning from the original data into the testing data
#random_sate sets random number generator speed
cardtrans_train, cardtrans_test = train_test_split(cardtrans, test_size = 0.20, random_state = 7)

#confirm partitining
print("The shape of the original dataset is", cardtrans.shape,
      "\nThe shape of the partitioned trainign set is", cardtrans_train.shape,
      "\nThe shape of the partitioned testing dataset is", cardtrans_test.shape)

↗ The shape of the original dataset is (1000000, 8)
The shape of the partitioned trainign set is (800000, 8)
The shape of the partitioned testing dataset is (200000, 8)
```

The training dataset is 80% of the original dataset and the testing dataset is 20% of the original dataset.

Check the training data set for the partition

Numerical variables: Two sample t-test for the difference in means

Categorical variable with two classes: Z-test for the difference in proportions

Categorical variables with more than two classes: test for the homogeneity of proportions

```
#import needed libraries
from scipy import stats #for t-test
import statsmodels.stats.proportion as smp #for test for homogeneity of proportions
```

distance_from_home, distance_from_last_transaction, and ratio_to_meadian_purchase_price are numerical variables. Check partitioning using t-test

```
#Perform the two-sample t-test distance_from_home
#(OpenAI, 2025a)[for reference page Gemini 3/27/2025 https://www.google.com/search?client=firefox-b-1-d&q=two+sample+t+test+in+python]
t_statistic, p_value = stats.ttest_ind(cardtrans_test['distance_from_home'], cardtrans_train['distance_from_home'])
```

```
#Print the results
print("T-statistic:", t_statistic)
print("P-value:", p_value)
```

```
#Check if the p-value is less than the significance level (e.g., 0.05)
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference between the means of the two groups.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference between the means of the two groups.")
```

```
↗ T-statistic: 1.6093572694997533
P-value: 0.10753855876007543
Fail to reject the null hypothesis: There is no significant difference between the means of the two groups.
```

```
#Perform the two-sample t-test distance_from_last_transaction (OpenAI, 2025)
t_statistic, p_value = stats.ttest_ind(cardtrans_test['distance_from_last_transaction'], cardtrans_train['distance_from_last_transaction'])
```

```
#Print the results
print("T-statistic:", t_statistic)
print("P-value:", p_value)
```

```
#Check if the p-value is less than the significance level (e.g., 0.05)
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference between the means of the two groups.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference between the means of the two groups.")
```

```

↗ T-statistic: -0.3491976786489459
P-value: 0.7269409823449284
Fail to reject the null hypothesis: There is no significant difference between the means of the two groups.

```

```

#Perform the two-sample t-test ratio_to_median_purchase_price (OpenAI, 2025)
t_statistic, p_value = stats.ttest_ind(cardtrans_test['ratio_to_median_purchase_price'], cardtrans_train['ratio_to_median_purchase_price'])

#Print the results
print("T-statistic:", t_statistic)
print("P-value:", p_value)

```

```

#Check if the p-value is less than the significance level (e.g., 0.05)
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference between the means of the two groups.")
else:
    print("Fail to reject the null hypothesis: There is no significant difference between the means of the two groups.")

```

```

↗ T-statistic: -0.15814964827070438
P-value: 0.8743389111122247
Fail to reject the null hypothesis: There is no significant difference between the means of the two groups.

```

The three numeric variables: distance_from_home, distance_from_last_transaction, and ratio_to_meadian_purchase_price failed to reject the null hypothesis. Therefore, p-values are less than the significance level and are statistically significant. Partitioning was successful.

Check partitioning by testing for homogeneity of proportions.

repeat_retailer, used_chip, used_pin_number, online_order, and fraud are categorical variables with more than two classes.

Outside the scope of the class, will come back later to figure out how to test homogeneity proportions of uneven test and training datasets, ie: count 200,000 and 800,000

✓ Balance the dataset for modeling

```

#Find percentage of data distribution to check relative frequencys
posper = fraud_counts[1]/(fraud_counts[1]+fraud_counts[0])
pospercent = round(posper*100, 2)
print("The percentage of positive fradulent creditcard use cases is", pospercent, "%")

```

```

↗ The percentage of positive fradulent creditcard use cases is 8.74 %

```

This dataset contains only 8.74 percent of positive cases. Balancing the training data may provide better modeling outcomes.

Increase the fraudulent records from 8.74% to 25%

```

#find the number of fraudulent records in the training set
#Get the distribution of values for 0 and 1
fraud_counts_train = cardtrans_train['fraud'].value_counts()
fraud_counts_train

```

```

↗
      count
fraud
0.0    730077
1.0     69923

dtype: int64

```

```

#Find percentage of data distribution to check relative frequencys
pospercent_train = fraud_counts_train[1]/(fraud_counts_train[1]+fraud_counts_train[0])
pospercent_train = round(pospercent_train*100, 2)
print("The percentage of positive fradulent creditcard use cases is", pospercent_train, "%")

```

```

↗ The percentage of positive fradulent creditcard use cases is 8.74 %

```

Same percentage of fraudulent cases as the original dataset

```
#Calculate records to resample
#rare + x = p(records + x)
resample_records = (0.25*(fraud_counts_train[1]+fraud_counts_train[0]) - fraud_counts_train[1]) / (1-0.25)
print("The number of records to resample to 25 percent fraudulent creditcard use cases is", resample_records)
```

➦ The number of records to resample to 25 percent fraudulent creditcard use cases is 173436.0

Test data should not be balanced

#Resampling the training dataset

```
#Isolate records to resample
to_resample = cardtrans_train.loc[cardtrans_train['fraud'] == 1.0]
```

```
#Sample from records of interest
our_resample = to_resample.sample(n = 173436, replace = True)
```

```
#add resampled records into original training dataset
cardtrans_train_rebal = pd.concat([cardtrans_train, our_resample])
```

```
#Check desired percentage of fraudulent records
rebal_counts = cardtrans_train_rebal['fraud'].value_counts()
```

```
#Find percentage of data distribution to check relative frequencies
pospercent_rebal = rebal_counts[1]/(rebal_counts[1]+rebal_counts[0])
pospercent_rebal = round(pospercent_rebal*100, 2)
print("The percentage of positive fraudulent creditcard use cases is", pospercent_rebal, "%")
```

➦ The percentage of positive fraudulent creditcard use cases is 25.0 %

Create a side-by-side bar chart to show class distribution before and after resampling

```
# Get original training class counts
orig_counts = cardtrans_train['fraud'].value_counts().sort_index()
# Get rebalanced training class counts
reb_counts = cardtrans_train_rebal['fraud'].value_counts().sort_index()
```

Combine into a DataFrame for easy plotting

```
dist_df = pd.DataFrame({
    'Before Resampling': orig_counts,
    'After Resampling': reb_counts
})
```

Plot

```
dist_df.plot(kind='bar', figsize=(8, 6))
plt.title('Fraud Class Distribution Before and After Resampling')
plt.xlabel('Fraud Label')
plt.ylabel('Count')
plt.xticks([0, 1], ['Non-Fraud (0)', 'Fraud (1)'], rotation=0)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```



Dataset balancing was successful

✓ Modeling

The Data Mining task is to create a Classification model to classify cases of fraudulent credit card use. This is a supervised learning method.

Decision Tree

Decision tree will be an appropriate base model for the features of this dataset. Since data features show non-linear relationships.

✓ Build a CART Decision Tree Using Python on training data

```
#Packages needed for CART Decision Tree
import statsmodels.tools.tools as stattools
from sklearn.tree import DecisionTreeClassifier, export_graphviz

#Save fraud as the independent variable
y = cardtrans_train_rebal[['fraud']]

#Save distance_from_home,distance_from_last_transaction,ratio_to_median_purchase_price,repeat_retailer, used_chip,used_pin_number, and online_order as dependent variables
X = cardtrans_train_rebal[['distance_from_home','distance_from_last_transaction',
                           'ratio_to_median_purchase_price','repeat_retailer',
                           'used_chip','used_pin_number', 'online_order']]

#Specify feature names
X_names = ['distance_from_home','distance_from_last_transaction',
            'ratio_to_median_purchase_price','repeat_retailer',
            'used_chip','used_pin_number', 'online_order']

y_names = [0,1]

#Run CART algorithm
cart01 = DecisionTreeClassifier(criterion = "gini", max_leaf_nodes=5).fit(X,y)
```

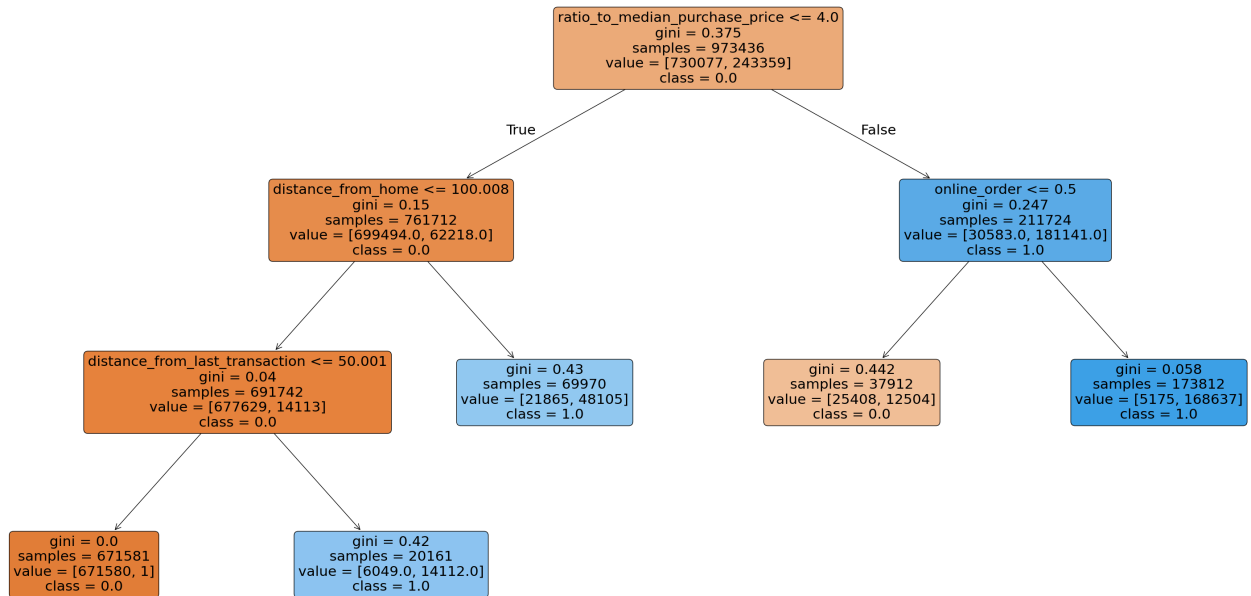
```
#Plot Decission Tree on training data  
from sklearn.tree import plot_tree
```

```
plt.figure(figsize=(40,20))  
plot_tree(cart01, feature_names = X.columns, class_names=y['fraud'].astype(str).unique(), filled=True, rounded = True)
```

```

[Text(0.5555555555555556, 0.875, 'ratio_to_median_purchase_price <= 4.0\ngini = 0.375\nsamples = 973436\nvalue = [730077, 243359]\nnclass = 0.0'),
Text(0.3333333333333333, 0.625, 'distance_from_home <= 100.008\ngini = 0.15\nsamples = 761712\nvalue = [699494.0, 62218.0]\nnclass = 0.0'),
Text(0.4444444444444444, 0.75, 'True '),
Text(0.2222222222222222, 0.375, 'distance_from_last_transaction <= 50.001\ngini = 0.04\nsamples = 691742\nvalue = [677629, 14113]\nnclass = 0.0'),
Text(0.1111111111111111, 0.125, 'gini = 0.0\nsamples = 671581\nvalue = [671580, 1]\nnclass = 0.0'),
Text(0.3333333333333333, 0.125, 'gini = 0.42\nsamples = 20161\nvalue = [6049.0, 14112.0]\nnclass = 1.0'),
Text(0.4444444444444444, 0.375, 'gini = 0.43\nsamples = 69970\nvalue = [21865, 48105]\nnclass = 1.0'),
Text(0.7777777777777778, 0.625, 'online_order <= 0.5\ngini = 0.247\nsamples = 211724\nvalue = [30583.0, 181141.0]\nnclass = 1.0'),
Text(0.6666666666666667, 0.75, ' False'),
Text(0.6666666666666666, 0.375, 'gini = 0.442\nsamples = 37912\nvalue = [25408, 12504]\nnclass = 0.0'),
Text(0.8888888888888888, 0.375, 'gini = 0.058\nsamples = 173812\nvalue = [5175, 168637]\nnclass = 1.0')]

```



✓ Build a CART Decision Tree Using Python on testing data for comparison

```
#Save fraud as the independent variable
y_test = cardtrans_test[['fraud']]

#Save distance_from_home,distance_from_last_transaction,ratio_to_median_purchase_price,repeat_retailer, used_chip,used_pin_number, and online_order
#as dependent variables
X_test = cardtrans_test[['distance_from_home','distance_from_last_transaction',
                        'ratio_to_median_purchase_price','repeat_retailer',
                        'used_chip','used_pin_number', 'online_order']]

#Run CART algorithm
cart02 = DecisionTreeClassifier(criterion = "gini", max_leaf_nodes=5).fit(X_test,y_test)

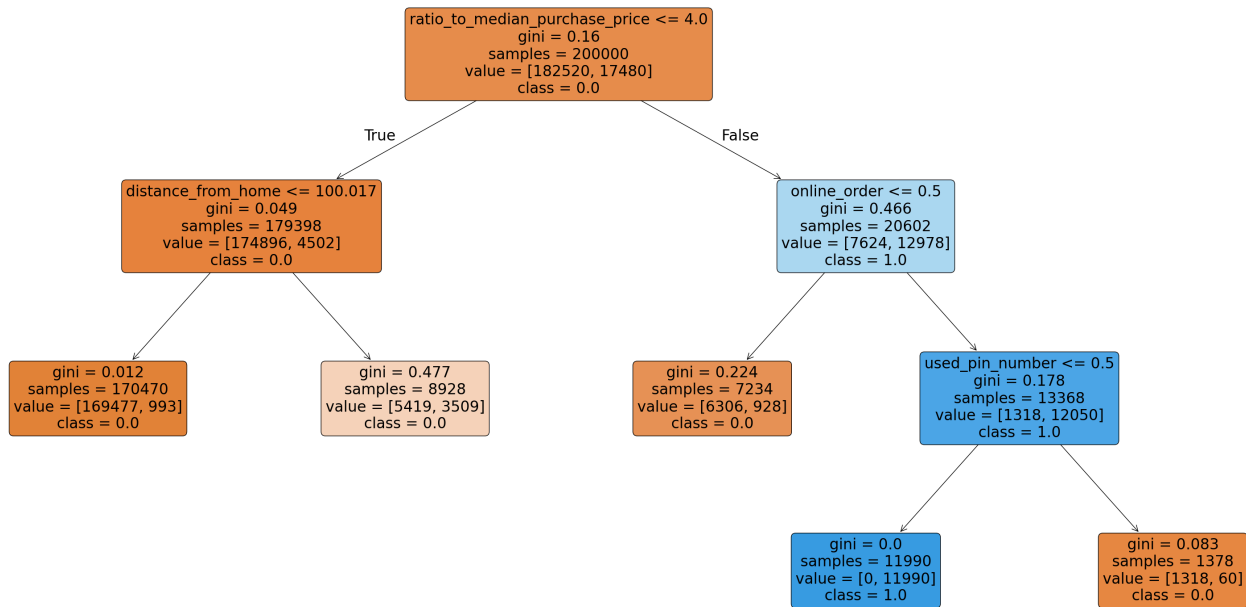
#Plot Decision Tree on testing data
from sklearn.tree import plot_tree

plt.figure(figsize=(40,20))
plot_tree(cart02, feature_names = X.columns, class_names=y['fraud'].astype(str).unique(), filled=True, rounded = True)
```

```

[Text(0.4444444444444444, 0.875, 'ratio_to_median_purchase_price <= 4.0\ngini = 0.16\nsamples = 200000\nvalue = [182520, 17480]\nnclass = 0.0'),
Text(0.2222222222222222, 0.625, 'distance_from_home <= 100.017\ngini = 0.049\nsamples = 179398\nvalue = [174896, 4502]\nnclass = 0.0'),
Text(0.3333333333333333, 0.75, 'True '),
Text(0.1111111111111111, 0.375, 'gini = 0.012\nsamples = 170470\nvalue = [169477, 993]\nnclass = 0.0'),
Text(0.3333333333333333, 0.375, 'gini = 0.477\nsamples = 8928\nvalue = [5419, 3509]\nnclass = 0.0'),
Text(0.6666666666666666, 0.625, 'online_order <= 0.5\ngini = 0.466\nsamples = 20602\nvalue = [7624, 12978]\nnclass = 1.0'),
Text(0.5555555555555556, 0.75, ' False'),
Text(0.5555555555555556, 0.375, 'gini = 0.224\nsamples = 7234\nvalue = [6306, 928]\nnclass = 0.0'),
Text(0.7777777777777778, 0.375, 'used_pin_number <= 0.5\ngini = 0.178\nsamples = 13368\nvalue = [1318, 12050]\nnclass = 1.0'),
Text(0.6666666666666666, 0.125, 'gini = 0.0\nsamples = 11990\nvalue = [0, 11990]\nnclass = 1.0'),
Text(0.8888888888888888, 0.125, 'gini = 0.083\nsamples = 1378\nvalue = [1318, 60]\nnclass = 0.0')]

```



Random Forest

✓ Build a Random Forest Using Python on training data

```
#Load libraries for Random Forest Model
from sklearn.ensemble import RandomForestClassifier

#Formate variables as one-dimensional numpy arrays
rfy = np.ravel(y)

#Create a Random Forest
rf01 = RandomForestClassifier(n_estimators = 100, criterion="gini").fit(X,rfy)
```

✓ Build a Random Forest Using Python on testing data for comparison

```
#Formate variables as one-dimensional numpy arrays
rfy_test = np.ravel(y_test)

#Create a Random Forest
rf02 = RandomForestClassifier(n_estimators = 100, criterion="gini").fit(X_test,rfy_test)
```

Naive Bayes

✓ Naive Bayes Model Using Python on training data

```
#Import required libraries for Naive Bayes
from sklearn.naive_bayes import MultinomialNB
import statsmodels.tools.tools as stattools

#Run Naive Bayes algorithm
nb = MultinomialNB().fit(X, rfy)
```

✓ Naive Bayes Model Using Python on test data

```
#Run Naive Bayes algorithm
nb_test = MultinomialNB().fit(X_test, rfy_test)
```

Neural Networks

✓ Neural Network Model Using Python on training data

```
#Load Libraries for Neural Network modeling
from keras.models import Sequential
from keras.layers import Dense

#Separate training and response variables
X_Neu = pd.DataFrame(cardtrans_train_rebal[['distance_from_home','distance_from_last_transaction',
                                             'ratio_to_median_purchase_price','repeat_retailer',
                                             'used_chip','used_pin_number', 'online_order']])
y_Neu = pd.DataFrame(cardtrans_train_rebal[['fraud']])
```

```
#View independent variable
X_Neu.head()
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_price	repeat_retailer	used_chip	used_pin_number
648911	7.719143	0.153094	2.207072	1.0	0.0	0.0
402969	4.096257	8.600147	1.554251	1.0	0.0	0.0
327515	1.100761	22.548549	0.040264	0.0	0.0	0.0
913774	28.187642	3.840116	1.040529	1.0	0.0	1.0
875277	6.081452	1.908698	9.460431	1.0	0.0	0.0

Normalize data to a range of 0-1 using Min/Max Scaler

```
#Create definition for Min/Max Scaler equation
def normal(x):
    normalized = (X_Neu[x] - X_Neu[x].min()) / (X_Neu[x].max() - X_Neu[x].min())
    return normalized

home_nor = normal('distance_from_home')
trans_nor = normal('distance_from_last_transaction')
pri_nor = normal('ratio_to_median_purchase_price')
```

```
#Display normalized data
print(home_nor.head(), '\n', trans_nor.head(), '\n', pri_nor.head())
```

```
648911    0.000877
402969    0.000464
327515    0.000123
913774    0.003209
875277    0.000690
Name: distance_from_home, dtype: float64
648911    0.000013
402969    0.000726
327515    0.001903
913774    0.000324
875277    0.000161
Name: distance_from_last_transaction, dtype: float64
648911    0.008220
402969    0.005782
327515    0.000129
913774    0.003864
875277    0.035305
Name: ratio_to_median_purchase_price, dtype: float64
```

Convert variables into 1-dimensional numpy array

```
#Find Variable types
```

```
#Independent variables
print(type(home_nor))
print(type(trans_nor))
print(type(pri_nor))
print(type(X_Neu['repeat_retailer']))
print(type(X_Neu['used_chip']))
print(type(X_Neu['used_pin_number']))
print(type(X_Neu['online_order']))
```

```
#Dependent variable
print(type(y_Neu))
```

```
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>
```

```
#Convert to 1-dimensional numpy array
def convert_num(n):
    a = np.array(n).reshape(-1,1)
```

```

return a

dis_home = convert_num(home_nor)
dis_transaction = convert_num(trans_nor)
ratio_price = convert_num(pri_nor)
rep_retailer = convert_num(X_Neu['repeat_retailer'])
us_chip = convert_num(X_Neu['used_chip'])
us_pin = convert_num(X_Neu['used_pin_number'])
on_order = convert_num(X_Neu['online_order'])

y_1d = convert_num(y_Neu)

#Redefine input features and output variables
X_Neu = np.concatenate((dis_home, dis_transaction, ratio_price, rep_retailer, us_chip, us_pin, on_order), axis=1)
y_Neu = y_1d

#Define the neural network architecture
model = Sequential()
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

#Compile the keras model
model.compile(loss='binary_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])

#Fit neural network model to dataset
model.fit(X_Neu, y_Neu, epochs=30, batch_size=32)

```

```

↩ Epoch 2/30
30420/30420 ————— 55s 2ms/step - accuracy: 0.7637 - loss: 0.4408
Epoch 3/30
30420/30420 ————— 84s 2ms/step - accuracy: 0.8269 - loss: 0.3777
Epoch 4/30
30420/30420 ————— 83s 2ms/step - accuracy: 0.8886 - loss: 0.3004
Epoch 5/30
30420/30420 ————— 80s 2ms/step - accuracy: 0.9140 - loss: 0.2505
Epoch 6/30
30420/30420 ————— 83s 2ms/step - accuracy: 0.9249 - loss: 0.2214
Epoch 7/30
30420/30420 ————— 56s 2ms/step - accuracy: 0.9310 - loss: 0.2006
Epoch 8/30
30420/30420 ————— 82s 2ms/step - accuracy: 0.9333 - loss: 0.1896
Epoch 9/30
30420/30420 ————— 82s 2ms/step - accuracy: 0.9349 - loss: 0.1819
Epoch 10/30
30420/30420 ————— 55s 2ms/step - accuracy: 0.9369 - loss: 0.1766
Epoch 11/30
30420/30420 ————— 55s 2ms/step - accuracy: 0.9378 - loss: 0.1734
Epoch 12/30
30420/30420 ————— 57s 2ms/step - accuracy: 0.9395 - loss: 0.1688
Epoch 13/30
30420/30420 ————— 81s 2ms/step - accuracy: 0.9397 - loss: 0.1672
Epoch 14/30
30420/30420 ————— 82s 2ms/step - accuracy: 0.9409 - loss: 0.1643
Epoch 15/30
30420/30420 ————— 81s 2ms/step - accuracy: 0.9416 - loss: 0.1614
Epoch 16/30
30420/30420 ————— 82s 2ms/step - accuracy: 0.9423 - loss: 0.1577
Epoch 17/30
30420/30420 ————— 81s 2ms/step - accuracy: 0.9431 - loss: 0.1542
Epoch 18/30
30420/30420 ————— 83s 2ms/step - accuracy: 0.9447 - loss: 0.1509
Epoch 19/30
30420/30420 ————— 82s 2ms/step - accuracy: 0.9447 - loss: 0.1497
Epoch 20/30
30420/30420 ————— 83s 2ms/step - accuracy: 0.9455 - loss: 0.1475
Epoch 21/30
30420/30420 ————— 57s 2ms/step - accuracy: 0.9461 - loss: 0.1464
Epoch 22/30
30420/30420 ————— 83s 2ms/step - accuracy: 0.9466 - loss: 0.1442
Epoch 23/30
30420/30420 ————— 81s 2ms/step - accuracy: 0.9473 - loss: 0.1423
Epoch 24/30
30420/30420 ————— 57s 2ms/step - accuracy: 0.9474 - loss: 0.1418
Epoch 25/30
30420/30420 ————— 59s 2ms/step - accuracy: 0.9482 - loss: 0.1394

```

```
Epoch 27/30
30420/30420 ————— 81s 2ms/step - accuracy: 0.9498 - loss: 0.1361
Epoch 28/30
30420/30420 ————— 57s 2ms/step - accuracy: 0.9494 - loss: 0.1354
Epoch 29/30
30420/30420 ————— 80s 2ms/step - accuracy: 0.9507 - loss: 0.1329
Epoch 30/30
30420/30420 ————— 82s 2ms/step - accuracy: 0.9513 - loss: 0.1314
```

✓ Neural Network Model Using Python on test data

#Separate training and response variables

```
X_Neu_test = pd.DataFrame(cardtrans_test[['distance_from_home','distance_from_last_transaction',
                                           'ratio_to_median_purchase_price','repeat_retailer',
                                           'used_chip','used_pin_number', 'online_order']])
y_Neu_test = pd.DataFrame(cardtrans_test[['fraud']])
```

#View independent variable

```
X_Neu_test.head()
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purchase_price	repeat_retailer	used_chip	used_pin_number
330111	114.594794	0.956496	2.718907	1.0	0.0	0.0
663520	0.521859	0.304829	0.848831	0.0	0.0	0.0
525009	12.679726	0.697291	0.975954	1.0	0.0	0.0
199449	3.134994	0.749451	0.610983	1.0	1.0	0.0
978049	3.549230	0.116309	1.913200	1.0	0.0	0.0

#Create definition for Min/Max Scaler equation

```
def normal_test (x):
    normalized = (X_Neu_test[x] - X_Neu_test[x].min()) / (X_Neu_test[x].max() - X_Neu_test[x].min())
    return normalized
```

```
home_nor_test = normal_test('distance_from_home')
```

```
trans_nor_test = normal_test('distance_from_last_transaction')
```

```
pri_nor_test = normal_test('ratio_to_median_purchase_price')
```

#Display normalized data

```
print(home_nor_test.head(), '\n',trans_nor_test.head(), '\n',pri_nor_test.head())
```

```
330111    0.010777
663520    0.000049
525009    0.001192
199449    0.000294
978049    0.000333
Name: distance_from_home, dtype: float64
330111    0.000325
663520    0.000104
525009    0.000237
199449    0.000255
978049    0.000039
Name: distance_from_last_transaction, dtype: float64
330111    0.028977
663520    0.009014
525009    0.010371
199449    0.006475
978049    0.020376
Name: ratio_to_median_purchase_price, dtype: float64
```

#Find Variable types

#Independent variables

```
print(type(home_nor_test))
print(type(trans_nor_test))
print(type(pri_nor_test))
print(type(X_Neu_test['repeat_retailer']))
print(type(X_Neu_test['used_chip']))
print(type(X_Neu_test['used_pin_number']))
print(type(X_Neu_test['online_order']))
```

#Dependent variable

```
print(type(y_Neu_test))
```

```

<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>

```

```

#Convert to 1-dimensional numpy array
dis_home_test = convert_num(home_nor_test)
dis_transaction_test = convert_num(trans_nor_test)
ratio_price_test = convert_num(pri_nor_test)
rep_retailer_test = convert_num(X_Neu_test['repeat_retailer'])
us_chip_test = convert_num(X_Neu_test['used_chip'])
us_pin_test = convert_num(X_Neu_test['used_pin_number'])
on_order_test = convert_num(X_Neu_test['online_order'])

y_1d_test = convert_num(y_Neu_test)

```

```

#Redefine input features and output variables
X_Neu_test = np.concatenate((dis_home_test, dis_transaction_test, ratio_price_test, rep_retailer_test, us_chip_test, us_pin_test, on_order_t
y_Neu_test = y_1d_test

```

```

#Fit neural network model to dataset
model.fit(X_Neu_test, y_Neu_test, epochs=30, batch_size=32)

```

```

Epoch 2/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9687 - loss: 0.1002
Epoch 3/30
6250/6250 ————— 11s 2ms/step - accuracy: 0.9698 - loss: 0.0944
Epoch 4/30
6250/6250 ————— 21s 2ms/step - accuracy: 0.9709 - loss: 0.0912
Epoch 5/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9709 - loss: 0.0886
Epoch 6/30
6250/6250 ————— 19s 2ms/step - accuracy: 0.9725 - loss: 0.0859
Epoch 7/30
6250/6250 ————— 22s 2ms/step - accuracy: 0.9726 - loss: 0.0860
Epoch 8/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9730 - loss: 0.0844
Epoch 9/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9731 - loss: 0.0827
Epoch 10/30
6250/6250 ————— 21s 2ms/step - accuracy: 0.9730 - loss: 0.0833
Epoch 11/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9735 - loss: 0.0807
Epoch 12/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9740 - loss: 0.0791
Epoch 13/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9736 - loss: 0.0785
Epoch 14/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9743 - loss: 0.0774
Epoch 15/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9742 - loss: 0.0774
Epoch 16/30
6250/6250 ————— 21s 2ms/step - accuracy: 0.9747 - loss: 0.0774
Epoch 17/30
6250/6250 ————— 21s 2ms/step - accuracy: 0.9760 - loss: 0.0728
Epoch 18/30
6250/6250 ————— 19s 2ms/step - accuracy: 0.9758 - loss: 0.0728
Epoch 19/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9765 - loss: 0.0706
Epoch 20/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9772 - loss: 0.0708
Epoch 21/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9769 - loss: 0.0688
Epoch 22/30
6250/6250 ————— 20s 2ms/step - accuracy: 0.9763 - loss: 0.0702
Epoch 23/30
6250/6250 ————— 11s 2ms/step - accuracy: 0.9768 - loss: 0.0684
Epoch 24/30
6250/6250 ————— 11s 2ms/step - accuracy: 0.9758 - loss: 0.0701
Epoch 25/30
6250/6250 ————— 21s 2ms/step - accuracy: 0.9771 - loss: 0.0692
Epoch 26/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9761 - loss: 0.0706
Epoch 27/30

```

```

6250/6250 ————— 12s 2ms/step - accuracy: 0.9772 - loss: 0.0673
Epoch 29/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9769 - loss: 0.0678
Epoch 30/30
6250/6250 ————— 12s 2ms/step - accuracy: 0.9765 - loss: 0.0677

```

Model Predictions and Evaluations

#Make model predictions for Decision Tree

```

# trainng and test datasets
pred_DT_Tr = cart01.predict(X)
pred_DT_Te = cart02.predict(X_test)

```

#Make model predictions for Random Forest

```

# Training and test dataset
pred_RF_Tr = rf01.predict(X)
pred_RF_Te = rf02.predict(X_test)

```

#Make model predictions for Naive Bayes

```

pred_NB_Tr = nb.predict(X)
pred_NB_Te = nb_test.predict(X_test)

```

#Make model predictions for Neural Networks

```

pred_NN_Tr = (model.predict(X_Neu) > 0.5).astype(int)
pred_NN_Te = (model.predict(X_Neu_test) > 0.5).astype(int)

```

```

→ 30420/30420 ————— 35s 1ms/step
6250/6250 ————— 7s 1ms/step

```

Confusion Matrix/ Performance Metric

```

#Import confusion matrix library
from sklearn.metrics import confusion_matrix

```

Decision Tree

```

#Decision Tree Training Confusion Matrix
cm_DT_Tr = confusion_matrix(y, pred_DT_Tr)
cm_DT_Tr

```

```

→ array([[696988, 33089],
        [12505, 230854]])

```



```
#Define False Negative, False Positice, True Negative, True Positive
TN_DT_Tr = cm_DT_Tr[0][0]
FP_DT_Tr = cm_DT_Tr[0][1]
FN_DT_Tr = cm_DT_Tr[1][0]
TP_DT_Tr = cm_DT_Tr[1][1]
```

```
#Define contingency table row and colum totals
```

```
# True Predicted Positive and Negative
```

```
# True Acutal Positive and Negative
```

```
TPN_DT_Tr = sum(cm_DT_Tr)[0]
```

```
TPP_DT_Tr = sum(cm_DT_Tr)[1]
```

```
TAN_DT_Tr = TN_DT_Tr + FP_DT_Tr
```

```
TAP_DT_Tr = FN_DT_Tr + TP_DT_Tr
```

```
#Caculate matrix total
```

```
GT_DT_TR = TN_DT_Tr + FP_DT_Tr + FN_DT_Tr + TP_DT_Tr
```

```
print('TN: ', TN_DT_Tr,
      '\nFP: ', FP_DT_Tr,
      '\nFN: ', FN_DT_Tr,
      '\nTP: ', TP_DT_Tr,
      '\nTPN: ', TPN_DT_Tr,
      '\nTPP: ', TPP_DT_Tr,
      '\nTAN: ', TAN_DT_Tr,
      '\nTAP: ', TAP_DT_Tr,
      '\nGT: ', GT_DT_TR)
```

```
↩ TN: 696988
   FP: 33089
   FN: 12505
   TP: 230854
   TPN: 709493
   TPP: 263943
   TAN: 730077
   TAP: 243359
   GT: 973436
```

```
#Caculate performance metrics
```

```
Accuracy_DT_TR = (TN_DT_Tr + TP_DT_Tr)/ GT_DT_TR
```

```
Error_DT_TR = 1 - Accuracy_DT_TR
```

```
Sensitivity_DT_TR = TP_DT_Tr/TAP_DT_Tr
```

```
Specificity_DT_TR = TN_DT_Tr/TAN_DT_Tr
```

```
Precision_DT_TR = TP_DT_Tr/TPP_DT_Tr
```

```
F_DT_TR = (2*(Precision_DT_TR*Sensitivity_DT_TR))/(Precision_DT_TR+Sensitivity_DT_TR)
```

```
#Decision Tree Testing Confusion Matrix
```

```
cm_DT_Te = confusion_matrix(y_test, pred_DT_Te)
```

```
cm_DT_Te
```

```
↩ array([[182520,    0],
         [ 5490, 11990]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
```

```
TN_DT_Te = cm_DT_Te[0][0]
```

```
FP_DT_Te = cm_DT_Te[0][1]
```

```
FN_DT_Te = cm_DT_Te[1][0]
```

```
TP_DT_Te = cm_DT_Te[1][1]
```

```
#Define contingency table row and colum totals
```

```
# True Predicted Positive and Negative
```

```
# True Acutal Positive and Negative
```

```
TPN_DT_Te = sum(cm_DT_Te)[0]
```

```
TPP_DT_Te = sum(cm_DT_Te)[1]
```

```
TAN_DT_Te = TN_DT_Te + FP_DT_Te
```

```
TAP_DT_Te = FN_DT_Te + TP_DT_Te
```

```
#Caculate matrix total
```

```
GT_DT_Te = TN_DT_Te + FP_DT_Te + FN_DT_Te + TP_DT_Te
```

```
#Caculate performance metrics
```

```
Accuracy_DT_Te = (TN_DT_Te + TP_DT_Te)/ GT_DT_Te
```

```
Error_DT_Te = 1 - Accuracy_DT_Te
```

```
Sensitivity_DT_Te = TP_DT_Te/TAP_DT_Te
```

```
Specificity_DT_Te = TN_DT_Te/TAN_DT_Te
```

```
Precision_DT_Te = TP_DT_Te/TPP_DT_Te
F_DT_Te = (2*(Precision_DT_Te*Sensitivity_DT_Te))/(Precision_DT_Te+Sensitivity_DT_Te)
```

Random Forest

```
#Random Forest Training Confusion Matrix
cm_RF_Tr = confusion_matrix(y, pred_RF_Tr)
cm_RF_Tr
```

```
array([[730077,    0],
       [    0, 243359]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
TN_RF_Tr = cm_RF_Tr[0][0]
FP_RF_Tr = cm_RF_Tr[0][1]
FN_RF_Tr = cm_RF_Tr[1][0]
TP_RF_Tr = cm_RF_Tr[1][1]
```

```
#Define contingency table row and colum totals
# True Predicted Positive and Negative
# True Acutal Positive and Negative
TPN_RF_Tr = sum(cm_RF_Tr)[0]
TPP_RF_Tr = sum(cm_RF_Tr)[1]
TAN_RF_Tr = TN_RF_Tr + FP_RF_Tr
TAP_RF_Tr = FN_RF_Tr + TP_RF_Tr
```

```
#Caculate matrix total
GT_RF_TR = TN_RF_Tr + FP_RF_Tr + FN_RF_Tr + TP_RF_Tr
```

```
#Caculate performance metrics
Accuracy_RF_TR = (TN_RF_Tr + TP_RF_Tr)/ GT_RF_TR
Error_RF_TR = 1 - Accuracy_RF_TR
Sensitivity_RF_TR = TP_RF_Tr/TAP_RF_Tr
Specificity_RF_TR = TN_RF_Tr/TAN_RF_Tr
Precision_RF_TR = TP_RF_Tr/TPP_RF_Tr
F_RF_TR = (2*(Precision_RF_TR*Sensitivity_RF_TR))/(Precision_RF_TR+Sensitivity_RF_TR)
```

```
#Random Forest Testing Confusion Matrix
cm_RF_Te = confusion_matrix(y_test, pred_RF_Te)
cm_RF_Te
```

```
array([[182520,    0],
       [    0, 17480]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
TN_RF_Te = cm_RF_Te[0][0]
FP_RF_Te = cm_RF_Te[0][1]
FN_RF_Te = cm_RF_Te[1][0]
TP_RF_Te = cm_RF_Te[1][1]
```

```
#Define contingency table row and colum totals
# True Predicted Positive and Negative
# True Acutal Positive and Negative
TPN_RF_Te = sum(cm_RF_Te)[0]
TPP_RF_Te = sum(cm_RF_Te)[1]
TAN_RF_Te = TN_RF_Te + FP_RF_Te
TAP_RF_Te = FN_RF_Te + TP_RF_Te
```

```
#Caculate matrix total
GT_RF_Te = TN_RF_Te + FP_RF_Te + FN_RF_Te + TP_RF_Te
```

```
#Caculate performance metrics
Accuracy_RF_Te = (TN_RF_Te + TP_RF_Te)/ GT_RF_Te
Error_RF_Te = 1 - Accuracy_RF_Te
Sensitivity_RF_Te = TP_RF_Te/TAP_RF_Te
Specificity_RF_Te = TN_RF_Te/TAN_RF_Te
Precision_RF_Te = TP_RF_Te/TPP_RF_Te
F_RF_Te = (2*(Precision_RF_Te*Sensitivity_RF_Te))/(Precision_RF_Te+Sensitivity_RF_Te)
```

Naive Bayes

```
#Naive Bayes Training Confusion Matrix
```

```
#Naive Bayes Training Confusion Matrix
cm_NB_Tr = confusion_matrix(y, pred_NB_Tr)
cm_NB_Tr
```

```
↪ array([[693378, 36699],
        [104377, 138982]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
TN_NB_Tr = cm_NB_Tr[0][0]
FP_NB_Tr = cm_NB_Tr[0][1]
FN_NB_Tr = cm_NB_Tr[1][0]
TP_NB_Tr = cm_NB_Tr[1][1]
```

```
#Define contingency table row and colum totals
# True Predicted Positive and Negative
# True Acutal Positive and Negative
TPN_NB_Tr = sum(cm_NB_Tr)[0]
TPP_NB_Tr = sum(cm_NB_Tr)[1]
TAN_NB_Tr = TN_NB_Tr + FP_NB_Tr
TAP_NB_Tr = FN_NB_Tr + TP_NB_Tr
```

```
#Caculate matrix total
GT_NB_TR = TN_NB_Tr + FP_NB_Tr + FN_NB_Tr + TP_NB_Tr
```

```
#Caculate performance metrics
Accuracy_NB_TR = (TN_NB_Tr + TP_NB_Tr)/ GT_NB_TR
Error_NB_TR = 1 - Accuracy_NB_TR
Sensitivity_NB_TR = TP_NB_Tr/TAP_NB_Tr
Specificity_NB_TR = TN_NB_Tr/TAN_NB_Tr
Precision_NB_TR = TP_NB_Tr/TPP_NB_Tr
F_NB_TR = (2*(Precision_NB_TR*Sensitivity_NB_TR))/(Precision_NB_TR+Sensitivity_NB_TR)
```

```
#Naive Bayes Testing Confusion Matrix
cm_NB_Te = confusion_matrix(y_test, pred_NB_Te)
cm_NB_Te
```

```
↪ array([[178769, 3751],
        [12262, 5218]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
TN_NB_Te = cm_NB_Te[0][0]
FP_NB_Te = cm_NB_Te[0][1]
FN_NB_Te = cm_NB_Te[1][0]
TP_NB_Te = cm_NB_Te[1][1]
```

```
#Define contingency table row and colum totals
# True Predicted Positive and Negative
# True Acutal Positive and Negative
TPN_NB_Te = sum(cm_NB_Te)[0]
TPP_NB_Te = sum(cm_NB_Te)[1]
TAN_NB_Te = TN_NB_Te + FP_NB_Te
TAP_NB_Te = FN_NB_Te + TP_NB_Te
```

```
#Caculate matrix total
GT_NB_Te = TN_NB_Te + FP_NB_Te + FN_NB_Te + TP_NB_Te
```

```
#Caculate performance metrics
Accuracy_NB_Te = (TN_NB_Te + TP_NB_Te)/ GT_NB_Te
Error_NB_Te = 1 - Accuracy_NB_Te
Sensitivity_NB_Te = TP_NB_Te/TAP_NB_Te
Specificity_NB_Te = TN_NB_Te/TAN_NB_Te
Precision_NB_Te = TP_NB_Te/TPP_NB_Te
F_NB_Te = (2*(Precision_NB_Te*Sensitivity_NB_Te))/(Precision_NB_Te+Sensitivity_NB_Te)
```

Neural Network

```
#Neural Network Training Confusion Matrix
cm_NN_Tr = confusion_matrix(y_Neu, pred_NN_Tr)
cm_NN_Tr
```

```
↪ array([[729715, 362],
        [191763, 51596]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
TN_NN_Tr = cm_NN_Tr[0][0]
FP_NN_Tr = cm_NN_Tr[0][1]
FN_NN_Tr = cm_NN_Tr[1][0]
TP_NN_Tr = cm_NN_Tr[1][1]

#Define contingency table row and colum totals
# True Predicted Positive and Negative
# True Acutal Positive and Negative
TPN_NN_Tr = sum(cm_NN_Tr)[0]
TPP_NN_Tr = sum(cm_NN_Tr)[1]
TAN_NN_Tr = TN_NN_Tr + FP_NN_Tr
TAP_NN_Tr = FN_NN_Tr + TP_NN_Tr

#Caculate matrix total
GT_NN_TR = TN_NN_Tr + FP_NN_Tr + FN_NN_Tr + TP_NN_Tr

#Caculate performance metrics
Accuracy_NN_Tr = (TN_NN_Tr + TP_NN_Tr)/ GT_NN_TR
Error_NN_Tr = 1 - Accuracy_NN_Tr
Sensitivity_NN_Tr = TP_NN_Tr/TAP_NN_Tr
Specificity_NN_Tr = TN_NN_Tr/TAN_NN_Tr
Precision_NN_Tr = TP_NN_Tr/TPP_NN_Tr
F_NN_Tr = (2*(Precision_NN_Tr*Sensitivity_NN_Tr))/(Precision_NN_Tr+Sensitivity_NN_Tr)
```

```
#Neural Network Testing Confusion Matrix
cm_NN_Te = confusion_matrix(y_Neu_test, pred_NN_Te)
cm_NN_Te
```

```
array([[181373, 1147],
       [ 3204, 14276]])
```

```
#Define False Negative, False Positice, True Negative, True Positive
TN_NN_Te = cm_NN_Te[0][0]
FP_NN_Te = cm_NN_Te[0][1]
FN_NN_Te = cm_NN_Te[1][0]
TP_NN_Te = cm_NN_Te[1][1]

#Define contingency table row and colum totals
# True Predicted Positive and Negative
# True Acutal Positive and Negative
TPN_NN_Te = sum(cm_NN_Te)[0]
TPP_NN_Te = sum(cm_NN_Te)[1]
TAN_NN_Te = TN_NN_Te + FP_NN_Te
TAP_NN_Te = FN_NN_Te + TP_NN_Te

#Caculate matrix total
GT_NN_Te = TN_NN_Te + FP_NN_Te + FN_NN_Te + TP_NN_Te

#Caculate performance metrics
Accuracy_NN_Te = (TN_NN_Te + TP_NN_Te)/ GT_NN_Te
Error_NN_Te = 1 - Accuracy_NN_Te
Sensitivity_NN_Te = TP_NN_Te/TAP_NN_Te
Specificity_NN_Te = TN_NN_Te/TAN_NN_Te
Precision_NN_Te = TP_NN_Te/TPP_NN_Te
F_NN_Te = (2*(Precision_NN_Te*Sensitivity_NN_Te))/(Precision_NN_Te+Sensitivity_NN_Te)
```

Print Performance Metrics for all models

```
#Accuracy
print('Accuracy:',
      '\n',
      '\nDecision Tree Training Dataset: ',Accuracy_DT_TR,
      '\nDecision Tree Test Dataset: ',Accuracy_DT_Te,
      '\n',
      '\nRandom Forest Training Dataset: ',Accuracy_RF_TR,
      '\nRandom Forest Test Dataset: ',Accuracy_RF_Te,
      '\n',
      '\nNaive Bayes Training Dataset: ', Accuracy_NB_TR,
      '\nNaive Bayes Test Dataset: ',Accuracy_NB_Te,
      '\n',
      '\nNeural Network Trainign Dataset: ', Accuracy_NN_Tr,
      '\nNeural Network Test Dataset: ', Accuracy_NN_Te)
```

➦ Accuracy:

```
Decision Tree Training Dataset: 0.9531617897838173
Decision Tree Test Dataset: 0.97255

Random Forest Training Dataset: 1.0
Random Forest Test Dataset: 1.0

Naive Bayes Training Dataset: 0.8550741908045316
Naive Bayes Test Dataset: 0.919935

Neural Network Trainign Dataset: 0.8026321196257381
Neural Network Test Dataset: 0.978245
```

#Error Rate

```
print('Error Rate:',
      '\n',
      '\nDecision Tree Training Dataset: ',Error_DT_TR,
      '\nDecision Tree Test Dataset: ',Error_DT_Te,
      '\n',
      '\nRandom Forest Training Dataset: ',Error_RF_TR,
      '\nRandom Forest Test Dataset: ',Error_RF_Te,
      '\n',
      '\nNaive Bayes Training Dataset: ', Error_NB_TR,
      '\nNaive Bayes Test Dataset: ',Error_NB_Te,
      '\n',
      '\nNeural Network Trainign Dataset: ', Error_NN_Tr,
      '\nNeural Network Test Dataset: ', Error_NN_Te)
```

➦ Error Rate:

```
Decision Tree Training Dataset: 0.04683821021618273
Decision Tree Test Dataset: 0.027449999999999974

Random Forest Training Dataset: 0.0
Random Forest Test Dataset: 0.0

Naive Bayes Training Dataset: 0.14492580919546838
Naive Bayes Test Dataset: 0.08006500000000005

Neural Network Trainign Dataset: 0.1973678803742619
Neural Network Test Dataset: 0.021754999999999997
```

#Sensitivity

```
print('Sensitivity:',
      '\n',
      '\nDecision Tree Training Dataset: ',Sensitivity_DT_TR,
      '\nDecision Tree Test Dataset: ',Sensitivity_DT_Te,
      '\n',
      '\nRandom Forest Training Dataset: ',Sensitivity_RF_TR,
      '\nRandom Forest Test Dataset: ',Sensitivity_RF_Te,
      '\n',
      '\nNaive Bayes Training Dataset: ', Sensitivity_NB_TR,
      '\nNaive Bayes Test Dataset: ',Sensitivity_NB_Te,
      '\n',
      '\nNeural Network Trainign Dataset: ', Sensitivity_NN_Tr,
      '\nNeural Network Test Dataset: ', Sensitivity_NN_Te)
```

➦ Sensitivity:

```
Decision Tree Training Dataset: 0.9486150091017796
Decision Tree Test Dataset: 0.6859267734553776

Random Forest Training Dataset: 1.0
Random Forest Test Dataset: 1.0

Naive Bayes Training Dataset: 0.5710986649353424
Naive Bayes Test Dataset: 0.29851258581235696

Neural Network Trainign Dataset: 0.21201599283363262
Neural Network Test Dataset: 0.8167048054919909
```

#Specificity

```
print('Specificity:',
      '\n',
```

```

'\nDecision Tree Training Dataset: ',Specificity_DT_TR,
'\nDecision Tree Test Dataset: ',Specificity_DT_Te,
'\n',
'\nRandom Forest Training Dataset: ',Specificity_RF_TR,
'\nRandom Forest Test Dataset: ',Specificity_RF_Te,
'\n',
'\nNaive Bayes Training Dataset: ', Specificity_NB_TR,
'\nNaive Bayes Test Dataset: ',Specificity_NB_Te,
'\n',
'\nNeural Network Trainign Dataset: ', Specificity_NN_Tr,
'\nNeural Network Test Dataset: ', Specificity_NN_Te)

```

 Specificity:

```

Decision Tree Training Dataset: 0.9546773833444966
Decision Tree Test Dataset: 1.0

Random Forest Training Dataset: 1.0
Random Forest Test Dataset: 1.0

Naive Bayes Training Dataset: 0.9497326994275946
Naive Bayes Test Dataset: 0.9794488275257506

Neural Network Trainign Dataset: 0.9995041618897733
Neural Network Test Dataset: 0.9937157571772957

```

#Precision

```

print('Precision:',
      '\n',
      '\nDecision Tree Training Dataset: ',Precision_DT_TR,
      '\nDecision Tree Test Dataset: ',Precision_DT_Te,
      '\n',
      '\nRandom Forest Training Dataset: ',Precision_RF_TR,
      '\nRandom Forest Test Dataset: ',Precision_RF_Te,
      '\n',
      '\nNaive Bayes Training Dataset: ', Precision_NB_TR,
      '\nNaive Bayes Test Dataset: ',Precision_NB_Te,
      '\n',
      '\nNeural Network Trainign Dataset: ', Precision_NN_Tr,
      '\nNeural Network Test Dataset: ', Precision_NN_Te)

```

 Precision:

```

Decision Tree Training Dataset: 0.8746358115199115
Decision Tree Test Dataset: 1.0

Random Forest Training Dataset: 1.0
Random Forest Test Dataset: 1.0

Naive Bayes Training Dataset: 0.791104331145656
Naive Bayes Test Dataset: 0.5817816924963765

Neural Network Trainign Dataset: 0.9930328342122483
Neural Network Test Dataset: 0.9256305517733255

```

#FScore

```

print('FScore',
      '\n',
      '\nDecision Tree Training Dataset: ',F_DT_TR,
      '\nDecision Tree Test Dataset: ',F_DT_Te,
      '\n',
      '\nRandom Forest Training Dataset: ',F_RF_TR,
      '\nRandom Forest Test Dataset: ',F_RF_Te,
      '\n',
      '\nNaive Bayes Training Dataset: ', F_NB_TR,
      '\nNaive Bayes Test Dataset: ',F_NB_Te,
      '\n',
      '\nNeural Network Trainign Dataset: ',F_NN_Tr,
      '\nNeural Network Test Dataset: ', F_NN_Te)

```

 FScore

```

Decision Tree Training Dataset: 0.9101245412003106
Decision Tree Test Dataset: 0.8137088564642009

Random Forest Training Dataset: 1.0
Random Forest Test Dataset: 1.0

```

Naive Bayes Training Dataset: 0.6633352424589538

Naive Bayes Test Dataset: 0.3945706832016335

Neural Network Trainign Dataset: 0.34942790289756426

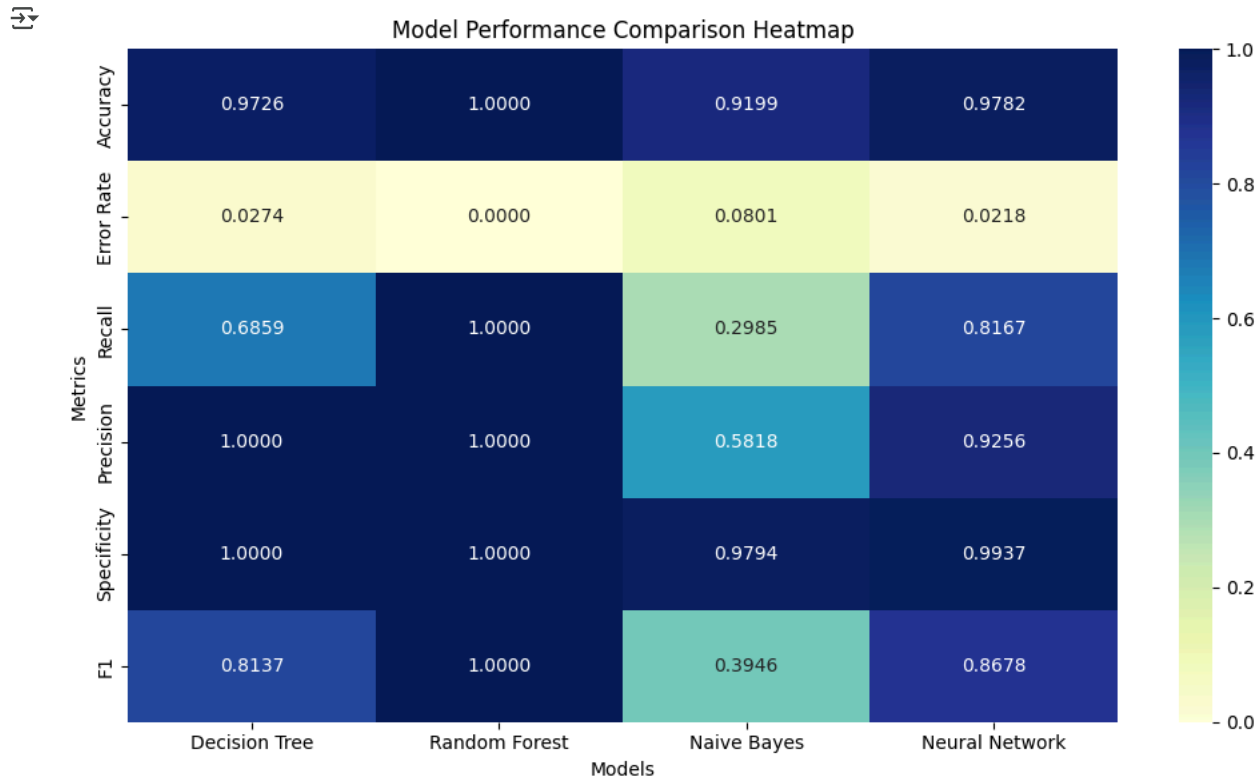
Neural Network Test Dataset: 0.8677628179801234

✓ Evaluation Metrics Table

```
# Performance Metrics
model_metrics = {
    "Metric": ["Accuracy", "Error Rate", "Recall", "Precision", "Specificity", "F1"],
    "Decision Tree": [Accuracy_DT_Te, Error_DT_Te, Sensitivity_DT_Te, Precision_DT_Te, Specificity_DT_Te, F_DT_Te],
    "Random Forest": [Accuracy_RF_Te, Error_RF_Te, Sensitivity_RF_Te, Precision_RF_Te, Specificity_RF_Te, F_RF_Te],
    "Naive Bayes": [Accuracy_NB_Te, Error_NB_Te, Sensitivity_NB_Te, Precision_NB_Te, Specificity_NB_Te, F_NB_Te],
    "Neural Network": [Accuracy_NN_Te, Error_NN_Te, Sensitivity_NN_Te, Precision_NN_Te, Specificity_NN_Te, F_NN_Te],
}

# Convert to DataFrame
metrics_df = pd.DataFrame(model_metrics)
metrics_df.set_index("Metric", inplace=True)

# Plot heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(metrics_df, annot=True, cmap="YlGnBu", fmt=".4f")
plt.title("Model Performance Comparison Heatmap")
plt.ylabel("Metrics")
plt.xlabel("Models")
plt.tight_layout()
plt.show()
```



✓ ROC Curve

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

#Predict probabilities (instead of classes)
rf_probs = rf02.predict_proba(X_test)[: , 1]
nb_probs = nb_test.predict_proba(X_test)[: , 1]
dt_probs = dt02.predict_proba(X_test)[: , 1]
```

```

nn_probs = model.predict(X_Neu_test).flatten() # Already sigmoid output

#Get ROC curve values
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_probs)
fpr_nb, tpr_nb, _ = roc_curve(y_test, nb_probs)
fpr_dt, tpr_dt, _ = roc_curve(y_test, dt_probs)
fpr_nn, tpr_nn, _ = roc_curve(y_Neu_test, nn_probs)

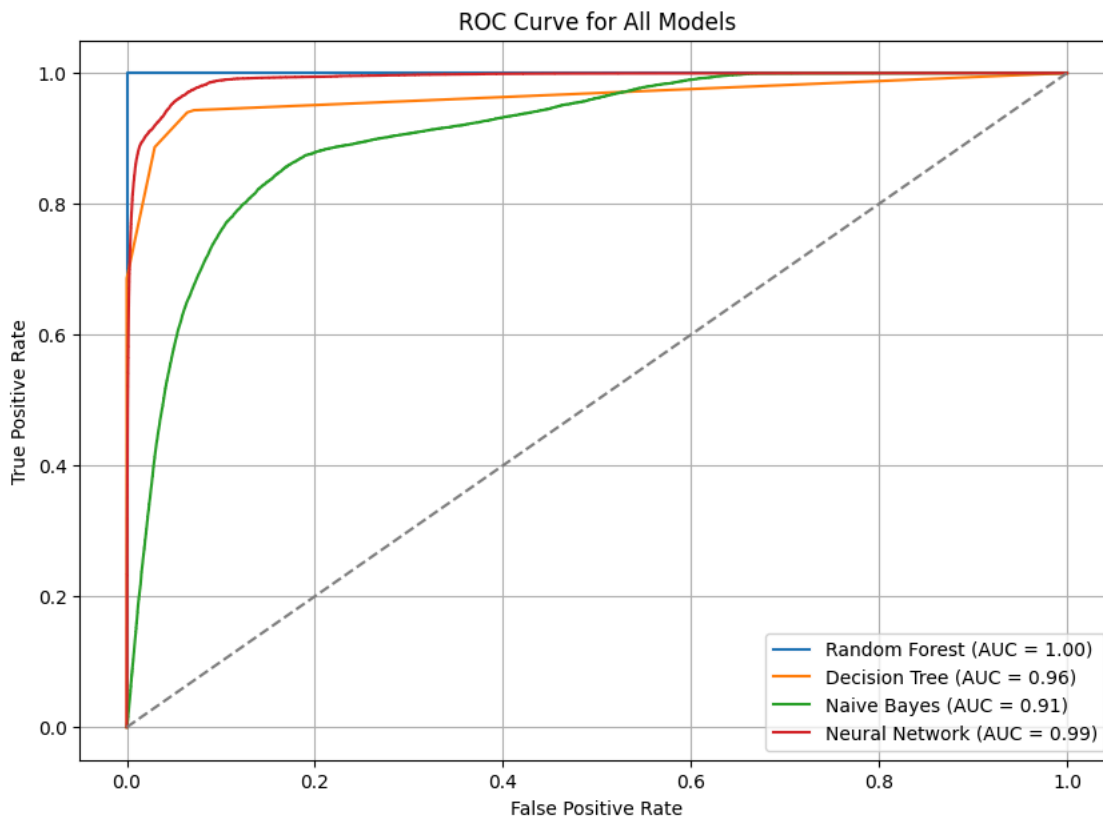
#Get AUC scores
auc_rf = auc(fpr_rf, tpr_rf)
auc_nb = auc(fpr_nb, tpr_nb)
auc_dt = auc(fpr_dt, tpr_dt)
auc_nn = auc(fpr_nn, tpr_nn)

#Plot all ROC curves
plt.figure(figsize=(10, 7))
plt.plot(fpr_rf, tpr_rf, label=f"Random Forest (AUC = {auc_rf:.2f})")
plt.plot(fpr_dt, tpr_dt, label=f"Decision Tree (AUC = {auc_dt:.2f})")
plt.plot(fpr_nb, tpr_nb, label=f"Naive Bayes (AUC = {auc_nb:.2f})")
plt.plot(fpr_nn, tpr_nn, label=f"Neural Network (AUC = {auc_nn:.2f})")
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve for All Models')
plt.legend()
plt.grid(True)
plt.show()

```

6250/6250 7s 1ms/step



✓ Cross-Validation, Feature Importance, and Confusion Matrix

```
!pip install scikeras
```

Collecting scikeras
 Downloading scikeras-0.13.0-py3-none-any.whl.metadata (3.1 kB)
 Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.11/dist-packages (from scikeras) (3.8.0)
 Requirement already satisfied: scikit-learn>=1.4.2 in /usr/local/lib/python3.11/dist-packages (from scikeras) (1.6.1)
 Requirement already satisfied: absl-py in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (1.4.0)
 Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (2.0.2)


```
Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (0.0.8)
Requirement already satisfied: h5py in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (3.13.0)
Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (0.14.1)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (0.4.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from keras>=3.2.0->scikeras) (24.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.4.2->scikeras) (1.14.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.4.2->scikeras) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.4.2->scikeras) (3.6)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/dist-packages (from optree->keras>=3.2.0->scikeras) (4.12.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.2.0->scikeras) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.2.0->scikeras) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.2.0->scikeras) (0.1.2)
Downloading scikeras-0.13.0-py3-none-any.whl (26 kB)
Installing collected packages: scikeras
Successfully installed scikeras-0.13.0
```

```
# Prepare training and test datasets
X = cardtrans_train_rebal[['distance_from_home', 'distance_from_last_transaction',
                           'ratio_to_median_purchase_price', 'repeat_retailer',
                           'used_chip', 'used_pin_number', 'online_order']]
y = cardtrans_train_rebal['fraud']

X_test = cardtrans_test[['distance_from_home', 'distance_from_last_transaction',
                           'ratio_to_median_purchase_price', 'repeat_retailer',
                           'used_chip', 'used_pin_number', 'online_order']]
y_test = cardtrans_test['fraud']
```

Decision Tree Classifier with Cross-Validation and Confusion Matrix

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.metrics import confusion_matrix

dt_model = DecisionTreeClassifier(max_leaf_nodes=5, random_state=42)
cv_dt = cross_val_score(dt_model, X, y, cv=5)
dt_model.fit(X, y)
dt_pred = dt_model.predict(X_test)
cm_dt = confusion_matrix(y_test, dt_pred)

print("Decision Tree Cross-Validation Scores:", cv_dt)
print("Confusion Matrix:\n", cm_dt)
```

Decision Tree Cross-Validation Scores: [0.95304795 0.95321208 0.95332508 0.95276521 0.95343295]

Confusion Matrix:

```
[[174259  8261]
 [  928 16552]]
```

Random Forest Cross-Validation, Confusion Matrix, and Feature Importance

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
cv_rf = cross_val_score(rf_model, X, y, cv=5)
rf_model.fit(X, y)
rf_pred = rf_model.predict(X_test)
cm_rf = confusion_matrix(y_test, rf_pred)

feature_importance = pd.Series(rf_model.feature_importances_, index=X.columns).sort_values(ascending=False)
print("Random Forest Cross-Validation Scores:", cv_rf)
print("Confusion Matrix:\n", cm_rf)
print("\nFeature Importance:\n", feature_importance)
```

Random Forest Cross-Validation Scores: [0.99998973 1. 0.99999486 1.]

Confusion Matrix:

```
[[182520  0]
 [  0 17480]]
```

Feature Importance:

```
ratio_to_median_purchase_price    0.588243
distance_from_home                 0.172348
online_order                      0.110359
distance_from_last_transaction     0.058315
used_pin_number                   0.035149
```